Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Authors: Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau

ABSTRACT

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and they often struggle on knowledge-intensive tasks, exhibiting hallucinations and failing to provide verifiable citations. In this paper, we explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) ? models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG-Sequence and RAG-Token models, which retrieve document passages using dense vector embeddings and condition a seq2seq generator on these passages. We evaluate on open-domain QA, question generation, and fact verification, setting new benchmarks on Natural Questions and TriviaQA.

1. Introduction & Parametric vs Non-Parametric Memory

Pre-trained neural language models act as implicit knowledge bases, storing facts directly within billions of synaptic weights (parametric memory). However, parametric memory suffers from critical flaws: it cannot be easily expanded or updated without full retraining, it cannot reliably provide provenance or page citations for its claims, and it is prone to producing plausible-sounding factual hallucinations. We combine parametric memory with non-parametric memory?a dense vector index of Wikipedia passages retrieved using Maximum Inner Product Search (MIPS).

2. Dense Passage Retrieval (DPR) Integration

The retrieval component utilizes Dense Passage Retriever (DPR) with dual bi-encoders: a question encoder q(x) and a passage encoder d(z) mapping text into 768-dimensional vectors. Retrieval probability p(z|x) is given by softmax(q(x)^T * d(z)). The top-k documents are retrieved efficiently using FAISS index.

3. RAG Architectures: RAG-Sequence vs RAG-Token

We formulate two RAG generation architectures conditioned on retrieved passages z:
1. RAG-Sequence: The model retrieves k passages and uses the same document to generate the entire target sequence y: p_seq(y|x) = sum_z p(z|x) * prod_i p(y_i|x, z, y_{1:i-1}).
2. RAG-Token: The model can draw upon different passages for each generated token: p_token(y|x) = prod_i sum_z p(z|x) * p(y_i|x, z, y_{1:i-1}). This allows the generator to synthesize content by combining multiple distinct knowledge sources across words.

4. Benchmark Results on Open-Domain QA

On Open-Domain Natural Questions, RAG achieves 44.5% exact match (EM), outperforming REALM (40.4%) and T5-11B (34.5%). On TriviaQA, RAG reaches 56.8% EM. Generation evaluations show RAG produces factual text with 3x fewer hallucinations than closed-book models.

5. Limitations and Retrieval Vulnerabilities

RAG architectures exhibit distinct failure modes: First, if the retriever fails to retrieve relevant passages in top-k, the generator struggles to answer correctly ('retrieval mismatch'). Second, conflicting or contradictory information across retrieved passages can confuse the generator. Third, joint fine-tuning of the retrieval index and generator is computationally expensive.

6. Research Gaps & Frontiers

Future priorities include iterative multi-hop retrieval, self-reflective retrieval triggering (Self-RAG), and adaptive chunk routing in massive vector stores.